

EchoMind 技术亮点详解

前言

有需要的可以下载pdf。但是请不要倒卖，这个项目我沉淀了很久，定价也不是很高，感谢🙏（最后有用的话记得多推荐一下😊）

本文档说明 EchoMind 当前实现中的六大核心技术亮点。每个亮点都对应真实代码模块，并能通过 API 或 Docker 环境观察到效果。

特别强调：端到端评测是 EchoMind 当前已经实现的核心能力之一，不是后续规划。系统通过 POST /eval/run 支持内置用例和自定义用例评测，覆盖意图识别 Accuracy/Macro-F1、真实 Agent 回复质量、LLM-as-Judge 四维评分、回归检测和优化建议生成。也就是说，EchoMind 的技术亮点不仅包括“会回答”，还包括“能自动评估回答质量并发现退化”。

当前版本重点实现：

能力	关键代码	可观察入口
三路融合意图识别	core/intent_recognizer.py	/chat 响应中的 intent
查询改写 + 重排 + fallback	mcp/tool_manager.py、 mcp/knowledge_base.py	/chat、/search、/knowledge/*
Redis + ChromaDB 三级记忆	memory/conversation_memory.py	多轮 /chat、Docker 中查看 ChromaDB
多 Agent 路由和自动并行协作	agents/agent_orchestrator.py	/chat 响应中的 agent_type
Monitor 自动降权闭环	monitor/performance_monitor.py	/monitor
端到端评测	evaluation/evaluator.py	/eval/run

总体架构

代码块

1 用户请求

```
2    -> api/main.py
3    -> MemoryManager 读取上下文
4        - Redis 工作记忆
5        - ChromaDB 情景记忆 episodic
6        - ChromaDB 用户画像 user_profile
7    -> IntentRecognizer 三路融合识别意图
8    -> MCPToolManager 检索 knowledge_base 并构建知识上下文
9    -> AgentOrchestrator 路由到专业 Agent
10   -> Agent 基于记忆 + 知识库上下文调用 LLM 生成回复
11   -> MemoryManager 写入记忆并异步更新用户画像
12   -> PerformanceMonitor 采集指标并反馈路由降权
```

亮点一：端到端意图识别

文件：[core/intent_recognizer.py](#)

核心问题：客服系统需要判断用户到底是在查询、投诉、退款、技术故障，还是要求转人工。单靠关键词容易误判，单靠 LLM 成本和延迟较高，因此 EchoMind 使用三路融合。

技术方案

代码块

```
1  用户消息 + 最近对话历史
2      |—— LLM 语义理解
3      |   - Few-shot 示例
4      |   - 最近 3 轮上下文
5      |   - 输出 intent / confidence / reasoning
6      |
7      |—— Embedding 相似度
8      |   - 官方模式：参与投票，权重 20%
9      |   - 当前 SDK 无远端 embedding 资源时：本地字符 n-gram 哈希向量兜底
10     |   - 第三方 base_url 模式：默认禁用，避免兼容 API 不支持 embedding
11     |
12     |—— Pattern 关键词匹配
13         - 零延迟兜底
14         - 覆盖退款、报错、转人工、投诉等高频表达
15
16         ↓
17  加权投票
18         ↓
19  意图 + 置信度 + 紧急度 + 实体
```

权重策略

场景	LLM	Embedding	Pattern
官方 API 模式	70%	20%	10%
第三方兼容 API 模式	85%	禁用	15%

工程细节

- **并行执行**：LLM 识别和 Embedding 识别通过 asyncio.gather 并行执行，关键词匹配同步完成。
- **Embedding 兜底**：如果客户端未来提供 embeddings.create，优先使用远端向量；当前 Anthropic SDK 没有该资源时，自动使用稳定的本地字符 n-gram 哈希向量。
- **低置信度保护**：投票得分低于阈值时降级为 OTHER，减少错误路由。
- **实体提取**：额外调用 LLM 提取 order_id、product、date、amount、error_code。
- **紧急度识别**：根据 “紧急” “马上” “今天” “转人工” 等关键词计算 LOW/MEDIUM/HIGH/CRITICAL。
- **在线学习**：learn() 可以把人工纠正样本加入模板，并清除对应 Embedding 缓存。

支持的意图

意图	说明	示例
query	查询信息	“我的订单什么时候到？”
complaint	投诉不满	“你们服务太差了！”
request	请求操作	“帮我取消订单”
technical	技术问题	“应用一直报 500 错误”
billing	账单/退款	“为什么扣了两次款？”
account	账户管理	“修改我的邮箱地址”
escalation	升级/转人工	“我要投诉，转人工！”
greeting	问候	“你好”
feedback	正面反馈	“服务很棒”

亮点二：MCP 工具调用框架 + RAG 知识库

文件：mcp/tool_manager.py、mcp/knowledge_base.py

核心问题：客服 Agent 不能只靠模型记忆回答业务问题，还需要查询业务知识库。同时，检索系统常见问题是召回不全、排序不准、工具异常影响主链路。

检索优化链路

代码块

```
1  用户查询：“退款需要多久”
2      ↓
3  1. LLM 查询改写
4      生成多个角度的子查询：
5      - “退款审核需要多久”
6      - “退款到账需要几天”
7      - “退款政策是什么”
8      ↓
9  2. 并行召回 ChromaDB
10     每个子查询同时检索 knowledge_base collection
11     ↓
12  3. 合并去重
13     按结果内容哈希去重
14     ↓
15  4. LLM 重排
16     根据原始问题对候选结果重新排序
17     ↓
18  5. 返回 Top-K
```

RAG 知识库实现

KnowledgeBase 使用 ChromaDB collection：

代码块

```
1  knowledge_base
```

能力包括：

- 启动时自动导入默认客服文档。
- /knowledge/add 批量导入文档。
- /knowledge/upload 上传 .txt、.md、.json 文档。
- 长文档自动切片，每片约 500 字。

- 查询时使用 ChromaDB 的 query_texts 做语义检索。

工具可靠性

MCPToolManager.call() 统一封装工具调用生命周期：

代码块

```
1  缓存检查
2      -> 熔断检查
3      -> 参数校验
4      -> asyncio.wait_for 超时控制
5      -> 调用 handler
6      -> 写入缓存
7      -> 可选 LLM 重排
8      -> 异常时 fallback
```

可靠性设计：

机制	作用
JSON Schema 参数校验	阻止错误参数进入工具 handler
TTL 缓存	相同参数复用结果，降低重复调用成本
Circuit Breaker	连续失败达到阈值后打开熔断，恢复窗口后半开探测
Timeout	工具级超时，避免单个工具拖垮请求
Fallback	工具不可用时返回可解释降级结果
ToolStats	记录成功率、延迟、连续失败次数，供 Monitor 使用

当前知识库 fallback

api/main.py 注册 knowledge_search 工具时配置了 fallback。当 ChromaDB 不可用或工具异常时，不会直接返回空错误，而是返回类似：

代码块

```
1  {
2    "title": "知识库降级结果",
3    "content": "知识库暂时不可用，未能完成语义检索。请稍后重试，或转人工客服确认。",
4    "fallback": true
5  }
```

接入 /chat 主链路

当前 /chat 不再只依赖 LLM 和对话记忆。主对话接口会在 Agent 执行前调用：

代码块

```
1  MCPToolManager.search_with_rewrite("knowledge_search", 用户消息, top_k=3)
```

然后把 Top-K 结果拼入 Agent 上下文：

代码块

```
1  [知识库检索结果]
2  1. 标题：退款政策
3     相关度：0.82
4     内容：退款申请提交后，系统会在 1-3 个工作日内审核...
5
6  请优先依据以上知识库内容回答；如果知识库内容不足，再结合通用客服能力说明。
```

业务收益：

收益	说明
降低幻觉	Agent 回复受知识库政策约束
知识可更新	通过 /knowledge/add 或 /knowledge/upload 更新文档即可影响 /chat
主链路闭环	RAG 不再只是 /search 演示，而是参与真实对话
可观测	/chat 响应新增 knowledge_used，可判断本次是否注入知识库上下文

亮点三：三级记忆管理

文件：`memory/conversation_memory.py`

核心问题：多轮客服对话既要记住当前会话，又不能把所有历史都塞进 prompt，还要能跨会话理解用户偏好。

三层记忆

代码块

```
1  [
2  |  工作记忆 Redis
3  |  - 当前会话最近消息
4  |  - 默认最多读取 20 条
5  |  - TTL 24 小时
6  |
7  |  ↓ 超过 15 条触发压缩
8  |
9  |  情景记忆 ChromaDB: episodic
10 |  - 压缩后的历史对话摘要
11 |  - 按 user_id 隔离
12 |  - query_texts 语义检索相关历史
13 |
14 |  ↓ 每轮对话后异步提炼
15 |
16 |  用户画像 ChromaDB: user_profile
17 |  - 用户偏好
18 |  - 常见实体
19 |  - 常见问题类型
20 |  ]
```

工作记忆

每次 /chat 回复后，系统会写入两条消息：

代码块

```
1  user: 用户原始消息
2  assistant: Agent 回复
```

Redis key 格式：

代码块

```
1   wm:{user_id}:{conv_id}
```

摘要 key 格式：

代码块

```
1   summary:{user_id}:{conv_id}
```

自动压缩

当工作记忆达到 COMPRESS_AT = 15 条时：

代码块

```
1   读取当前会话消息
2       ↓
3   保留最近 5 条
4       ↓
5   旧消息交给 LLM 生成 2-3 句摘要
6       ↓
7   摘要写入 Redis summary
8       ↓
9   摘要写入 ChromaDB episodic
10      ↓
11  工作记忆只保留最近 5 条
```

这样能避免 context 随着对话增长无限膨胀。

用户画像

每次 /chat 结束后，api/[main.py](#) 会异步执行：

代码块

```
1   asyncio.create_task(_memory.update_profile(req.user_id, conv_id))
```

update_profile() 会取最近 10 条消息，让 LLM 提炼：

代码块

```
1   {
2       "preferences": ["喜欢简洁回答", "经常咨询退款"],
3       "entities": {
4           "产品": ["会员服务"],
```



```
5     "问题类型": ["账单", "登录"]
6 }
7 }
```

这些画像被写入 ChromaDB 的 user_profile collection。下一次同一 user_id 请求时，get_context() 会读出画像，并由 MemoryContext.to_prompt_text() 拼进 Agent 背景信息。

上下文拼接效果

最终传给 Agent 的上下文结构类似：

代码块

```
1  [会话摘要]
2  用户之前咨询过退款，要求处理速度快。
3
4  [相关历史]
5  - 用户三天前反馈过同一订单延迟。
6
7  [用户画像]
8  {"preferences": ["快速响应"], "entities": {"问题类型": ["退款"]}}
9
10 [最近对话]
11 user: 我的退款到了吗？
12 assistant: 我来帮您确认退款进度。
```

亮点四：多 Agent 路由与编排

文件：[agents/agent_orchestrator.py](#)

核心问题：不同客服问题需要不同专业能力，系统需要把请求路由到合适 Agent，同时处理失败降级和复合问题协作。

Agent 类型

Agent	负责问题
GeneralAgent	通用咨询、问候、普通请求
TechnicalAgent	登录失败、崩溃、错误码、系统配置
BillingAgent	

	账单、退款、发票、订阅、账户相关
ESCALATION	转人工/升级占位，不直接调用 LLM Agent

三层路由

代码块

```
1  请求进入 Orchestrator
2      ↓
3  1. 意图路由
4      TECHNICAL -> TechnicalAgent
5      BILLING/ACCOUNT -> BillingAgent
6      ESCALATION -> 升级
7      其他 -> GeneralAgent
8      ↓
9  2. 性能路由
10     同类型有多个实例时，选择 routing_score 最高的实例
11     ↓
12  3. 降级路由
13     专属 Agent 不可用或执行失败时，降级到 GeneralAgent
```

routing_score

AgentStats.routing_score() 由三部分组成：

代码块

```
1  base_score = success_rate * 0.7 + latency_score * 0.3
2  routing_score = base_score * (1 - monitor_penalty)
```

其中：

- success_rate：Agent 成功率。
- latency_score：由平均延迟换算，延迟越低得分越高。
- monitor_penalty：Monitor 根据在线表现写回的降权系数，范围 0 到 0.9。

自动并行协作

当前 run() 主链路已经接入复合问题检测，不再只是手动调用 run_parallel()。

示例：

1470 他登录报错 401，而且这个月还重复扣款了”

系统会同时识别到技术关键词和账单关键词，自动并行调用：

代码块

```
1 TechnicalAgent + BillingAgent
```

然后把两个成功响应合并返回：

代码块

```
1 [technical]
2 ...
3
4 [billing]
5 ...
```

升级机制

以下情况会标记 `escalated = true`：

- 用户意图或紧急度触发升级。
- 用户包含强紧急表达。
- Agent 回复中出现“转人工”“人工客服”“无法处理”等关键词。

生产环境中可在该位置对接工单系统、客服队列或告警系统。

亮点五：Monitor 闭环监控与自动降权

文件：[monitor/performance_monitor.py](#)

核心问题：Agent 或工具在线表现变差时，系统不能只记录日志，还应影响后续路由，减少问题 Agent 被继续选中的概率。

监控闭环

代码块

```
1 Agent / Tool 执行请求
2   ↓
3 实时更新 AgentStats / ToolStats
4   ↓
```

```
5 PerformanceMonitor 每 10s 采集一次
6     ↓
7  阈值告警 + Z-score 异常检测
8     ↓
9  计算每个 Agent 的 routing_penalty
10    ↓
11  调用 Orchestrator.update_routing_penalties()
12    ↓
13  写回 AgentStats.monitor_penalty
14    ↓
15  下一次 _best_agent() 使用新的 routing_score
```

告警阈值

指标	阈值	级别
Agent 成功率	< 0.90	ERROR
工具成功率	< 0.95	WARNING
Agent 平均延迟	> 3000ms	WARNING
工具平均延迟	> 5000ms	ERROR

自动降权规则

Monitor 会把成功率和延迟转成 monitor_penalty:

代码块

```
1 success_rate < 0.90 -> 增加成功率惩罚
2 avg_ms > 3000      -> 增加延迟惩罚
3 最终 penalty 最大 0.9
```

这意味着某个 Agent 表现差时，它的 routing_score 会下降。同类型有多个实例时，Orchestrator 会自动选择表现更好的实例。

可观测输出

GET /monitor 会返回：

代码块

```
1 {
2   "agent_stats": {
```

```
3     "technical_0": {
4         "total": 20,
5         "success_rate": 0.85,
6         "avg_ms": 4200.0,
7         "monitor_penalty": 0.25,
8         "routing_score": 0.51
9     }
10 },
11 "tool_stats": {
12     "knowledge_search": {
13         "total": 10,
14         "success_rate": 0.9,
15         "avg_latency_ms": 120.5,
16         "consecutive_fails": 1,
17         "circuit_state": "closed"
18     }
19 },
20 "active_alerts": [],
21 "suggestions": []
22 }
```

Prometheus 支持

当前 FastAPI 应用通过 GET /metrics 暴露 Prometheus 指标，Docker Compose 中的 Prometheus 会抓取 echomind:8000/metrics。另外，如果单独设置 PROMETHEUS_PORT，Monitor 也可以启动独立的 Prometheus 指标端口。指标包括：

- Agent 成功率
- Agent 延迟
- 工具成功率
- 总请求数

亮点六：端到端 Agent 评测

文件：evaluation/[evaluator.py](#)

核心问题：Agent 系统不能只看单点功能是否可用，还需要评估端到端效果，包括意图识别、回复质量和回归风险。

评测内容

代码块

1 1. 意图识别评测

```
2      标注用例 -> IntentRecognizer -> Accuracy / Macro-F1
3
4      2. 端到端对话评测
5      测试问题 -> Orchestrator.run() -> 真实 Agent 回复
6
7      3. LLM-as-Judge
8      用 LLM 从四个维度给真实回复打分
9
10     4. 回归检测
11     当前结果和上一次评测结果比较, 退化超过 5% 标记 regression
12     支持通过 EVAL_BASELINE_PATH 持久化基线
13
14     5. 优化建议
15     根据低分指标输出具体优化方向
```

LLM-as-Judge 评分维度

维度	含义	分数
relevance	是否直接回应用户问题	0-1
accuracy	信息是否准确	0-1
completeness	是否完整解决需求	0-1
helpfulness	用户能否据此行动	0-1

API 使用

代码块

```
1  curl -X POST http://localhost:8000/eval/run
```

响应包含整体通过率、平均分、回归项、建议和逐条结果：

代码块

```
1  {
2    "pass_rate": 0.83,
3    "total": 5,
4    "passed": 4,
5    "avg_scores": {
6      "intent_accuracy": 0.875,
7      "relevance": 0.88,
```

```
8     "accuracy": 0.82,
9     "completeness": 0.79,
10    "helpfulness": 0.85
11  },
12  "regressions": [],
13  "recommendations": [
14    "意图识别准确率 < 90%: 增加 Few-shot 示例, 或对低 F1 的意图类别补充训练数据"
15  ],
16  "results": [
17    {
18      "test_id": "intent_recognition",
19      "passed": true,
20      "scores": {
21        "accuracy": 0.875,
22        "macro_f1": 0.82
23      }
24    }
25  ]
26 }
```

自定义和多轮评测

/eval/run 不传 body 时运行内置用例；传入 body 时可以覆盖意图识别用例和对话用例：

代码块

```
1  {
2    "intent_cases": [
3      {"message": "应用一直报错", "expected_intent": "technical"}
4    ],
5    "dialog_cases": [
6      {"question": "我的订单还没到"},
7      {"turns": ["你好, 我要退款", "订单号是 #12345", "退款多久能到账? "]}
8    ]
9  }
```

多轮用例会使用同一个 conv_id, 并把前序轮次作为评测历史上下文传入 Orchestrator 和 Judge。

ChromaDB 数据设计

EchoMind 中 ChromaDB 不是单一用途, 而是同时承担 RAG 知识库和长期记忆。

Collection	代码位置	写入来源	查询用途
------------	------	------	------

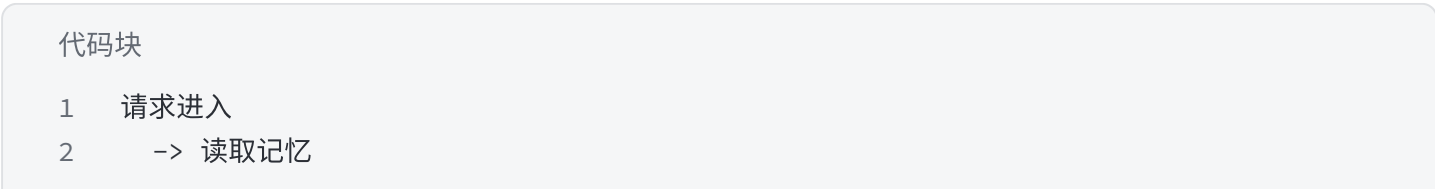
knowledge_base	mcp/knowledge_base.py	默认文档、/knowledge/add、/knowledge/upload	/search 检索业务知识
episodic	memory/conversation_memory.py	工作记忆压缩后的摘要	下一轮对话检索相关历史
user_profile	memory/conversation_memory.py	每次 /chat 后异步提炼	拼入 Agent 背景信息，支持个性化

查看方式见 wiki/完整使用指南.md 中 “在 Docker 中查看 ChromaDB 内容” 部分。

模块协作关系



端到端数据流：



- 3 -> 识别意图
- 4 -> 路由 Agent
- 5 -> 生成回复
- 6 -> 写入短期记忆
- 7 -> 异步更新用户画像
- 8 -> Monitor 采集表现
- 9 -> 下一次路由使用更新后的 routing_score